

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Разработка веб-приложения для автоматизации процессов
документооборота культурно-досуговых учреждений»

Выполнила: Крипак Ксения Романовна

Группа: 241-326

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Крипак Ксении Романовне,

обучающейся группы 241-326,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для автоматизации процессов докумен-
тооборота культурно-досуговых учреждений»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	11.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	14.04.2026	
Задача 1.3. Формулировка цели и задач работы	8	18.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	04.05.2026	
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	08.05.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	12.05.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	16.05.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка пользовательского интерфейса веб-приложения и реализация базовой структуры страниц	8	20.05.2026	
Задача 3.2. Реализация модуля аутентификации пользователей и разграничения прав доступа	7	26.05.2026	
Задача 3.3. Разработка CRUD-интерфейсов для управления мероприятиями и документами	7	30.05.2026	
Задача 3.4. Реализация механизма формирования и загрузки документов в форматах PDF и Excel	12	02.06.2026	
Задача 3.5. Разработка системы мониторинга статусов мероприятий и формирования отчетности	6	06.06.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	01.04.2026	
Задача 4.2. Деплой приложения на хостинг	4	13.06.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	13.06.2026	

Руководитель курсовой работы (проекта):
старший преподаватель кафедры «Инфокогнитивные технологии»

«___» _____ 2026 г. _____ А. С. Кружалов
(подпись)

Дата выдачи задания «01» апреля 2026 г.

Дата сдачи выполненной работы «___» _____ 2026 г.

Задание приняла к исполнению

«01» апреля 2026 г. _____ К. Р. Крипак
(подпись)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих систем электронного документооборота	6
1.2 Анализ программных средств и технологий разработки веб-приложений	7
1.3 Формулировка цели и задач работы	9
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	10
2.1 Анализ целевой аудитории	10
2.2 Описание функциональности приложения	11
2.3 Проектирование модели данных	13
2.4 Разработка макетов страниц	15
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	22
3.1 Разработка пользовательского интерфейса и реализация базовой структуры страниц	22
3.2 Реализация модуля аутентификации пользователей и разграничения прав доступа	24
3.3 Разработка CRUD-интерфейсов для управления мероприятиями и документами	26
3.4 Реализация механизма формирования и выгрузки отчётов в формате Excel	29
3.5 Разработка системы мониторинга статусов мероприятий и формирования отчётности	32

ВВЕДЕНИЕ

Актуальность темы. В условиях цифровой трансформации общества особую значимость приобретает автоматизация документооборота. Культурно-досуговые учреждения (дома культуры, театры, центры творчества) ежедневно обрабатывают значительные объемы документации, включая планы мероприятий, отчеты, договоры и внутренние служебные документы. Использование традиционных бумажных или частично автоматизированных методов ведения документации приводит к снижению эффективности работы, увеличению временных затрат на обработку информации и повышению риска ошибок. Разработка специализированных веб-приложений для автоматизации процессов документооборота позволяет повысить прозрачность управления, ускорить обработку данных и обеспечить централизованный доступ к информации. В качестве практической базы для анализа предметной области рассматривается деятельность МУ ЦКиД «Факел» г. Фрязино.

Степень разработанности проблемы. Вопросы автоматизации документооборота широко рассматриваются в научной и практической литературе. Существуют как универсальные системы электронного документооборота (например, «1С:Документооборот», «Directum», «DocsVision»), так и специализированные решения для отдельных отраслей. Однако большинство коммерческих систем характеризуется высокой стоимостью внедрения и избыточной функциональностью, не всегда соответствующей специфике культурно-досуговых учреждений. В то же время open-source решения требуют значительной доработки для адаптации под конкретные бизнес-процессы. Таким образом, задача разработки гибкого и адаптируемого веб-приложения для автоматизации документооборота в данной сфере остается актуальной.

Объект исследования — процессы организации и ведения документооборота в культурно-досуговых учреждениях (на примере МУ ЦКиД «Факел» г. Фрязино).

Предмет исследования — методы и программные средства автоматизации документооборота на основе веб-технологий.

Целью работы является проектирование и разработка веб-приложения для автоматизации документооборота в культурно-досуговых учреждениях на примере МУ ЦКиД «Факел» г. Фрязино.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих систем электронного документооборота

На современном этапе цифровизации организаций системы электронного документооборота (СЭД) являются ключевым инструментом автоматизации управленческой деятельности. Они обеспечивают централизованное хранение документов, контроль версий, маршрутизацию согласования и разграничение прав доступа. В таблице 1.1 представлен сравнительный анализ наиболее распространённых решений.

Таблица 1.1 – Сравнительный анализ систем электронного документооборота

Критерий сравнения	1С:Документооборот	Directum	Alfresco
Платформа и доступ	Desktop/Web, интеграция с 1С	Web, SaaS и On-premise	Web, кроссплатформенная
Управление документами	Полный цикл работы с документами, шаблоны, маршруты	Развитая маршрутизация и BPM	Гибкое хранение и версионирование
Ролевая модель	Гибкая настройка ролей и прав	Разграничение доступа и бизнес-роли	Настраиваемая модель доступа
Аналитика и отчёты	Расширенные отчёты и интеграция с учетными системами	BPM-аналитика процессов	Ограниченные встроенные средства
Гибкость и кастомизация	Требует доработок под специфику	Высокая, но сложная настройка	Высокая (open-source)
Ключевой недостаток	Высокая стоимость внедрения и сопровождения	Сложность внедрения для малых организаций	Требует технической доработки и поддержки

Анализ показал, что существующие системы ориентированы преимущественно на крупные организации и корпоративный сектор. Они обладают высокой функциональной сложностью и требуют значительных ресурсов на внедрение, сопровождение и обучение сотрудников работе с ними. Для культурно-досуговых учреждений, таких как МУ ЦКиД «Факел» г. Фрязино, характерны более простые, но специфические процессы документооборота, а также небольшие бюджеты на поддержание этих процессов. Это обосновывает необходимость разработки легковесного и адаптированного веб-приложения, ориентированного на реальные потребности пользователей.

1.2 Анализ программных средств и технологий разработки веб-приложений

Выбор технологического стека определялся требованиями к простоте разработки, скорости внедрения, масштабируемости и удобству сопровождения. Архитектура приложения строится по клиент-серверной модели. В таблицах 1.2, 1.3, 1.4 представлен сравнительный анализ различных технологий в рамках создания веб-приложения для автоматизации процессов документооборота в культурно-досуговых учреждениях.

Таблица 1.2 – Сравнительный анализ клиентских технологий

Технология	Преимущества	Недостатки	Итог
Чистый JS + HTML + CSS	Минимальный порог входа, простота разработки, отсутствие сложной сборки	Ограниченные возможности масштабирования интерфейса	Выбран - оптимален для учебного проекта и умеренной сложности UI
React	Компонентная архитектура, масштабируемость, развитая экосистема	Усложнение проекта, необходимость настройки сборки	Не выбран - избыточен для текущих требований
Vue.js	Простота интеграции, реактивность, удобство разработки	Меньшая экосистема по сравнению с React	Не выбран - не требуется при простой логике интерфейса

Таблица 1.3 – Сравнительный анализ серверных технологий

Технология	Преимущества	Недостатки	Итог
FastAPI	Высокая производительность, асинхронность, автоматическая документация API, простота разработки	Требуется разделение на API и frontend	Выбран - оптимален по балансу простоты и производительности
Django	«Из коробки» включает ORM, админ-панель, авторизацию	Избыточность и монолитность для небольшой системы	Не выбран - избыточен для проекта
Node.js + Express	Единый язык с frontend, гибкость	Требуется ручной организации архитектуры, нет автоматической документации	Не выбран - нет критического преимущества перед FastAPI

Таблица 1.4 – Сравнительный анализ систем управления базами данных

СУБД	Преимущества	Недостатки	Итог
PostgreSQL	Надёжность, транзакции, сложные запросы, поддержка связей	Требует проектирования схемы	Выбран - оптимален для структурированных данных
MySQL	Простота и распространённость	Ограниченная функциональность сложных запросов	Не выбран - уступает PostgreSQL
MongoDB	Гибкая схема, масштабируемость	Сложность обеспечения связей и целостности	Не выбран - не подходит для строгой структуры данных

Для реализации клиентской части приложения используются базовые веб-технологии: HTML, CSS и JavaScript. Данный подход позволяет реализовать пользовательский интерфейс без использования тяжеловесных фреймворков, обеспечивая простоту разработки и достаточную функциональность для решения поставленных задач. В процессе разработки использовались официальные спецификации и руководства по веб-стандартам [1].

Для реализации серверной части приложения выбрана платформа FastAPI, являющаяся современным фреймворком для разработки веб-приложений на языке Python. FastAPI обеспечивает высокую производительность за счёт асинхронной обработки запросов, а также предоставляет встроенную валидацию данных и автоматическую генерацию документации API [2]. Использование данного фреймворка позволяет эффективно реализовать REST-интерфейс для взаимодействия клиентской и серверной частей приложения.

Для хранения данных в приложении выбрана система управления базами данных PostgreSQL. Данная СУБД обеспечивает надежное хранение структурированных данных, поддержку транзакций и строгую целостность данных. Кроме того, PostgreSQL предоставляет широкие возможности для выполнения сложных запросов, индексации и масштабирования, что делает её подходящей для реализации систем документооборота [3]. Использование реляционной модели данных позволяет эффективно организовать хранение сущностей приложения, таких как пользователи, документы, роли и права доступа, а также установить между ними необходимые связи.

1.3 Формулировка цели и задач работы

На основе проведенного анализа существующих решений (см. подраздел 1.1) и выбора технологического стека (см. подраздел 1.2) была сформулирована цель исследования.

Целью работы является проектирование и разработка веб-приложения для автоматизации документооборота в культурно-досуговых учреждениях на примере МУ ЦКиД «Факел» г. Фрязино.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ предметной области и выявить особенности документооборота в культурно-досуговых учреждениях на примере МУ ЦКиД «Факел».
2. Выполнить сравнительный анализ существующих систем электронного документооборота.
3. Спроектировать архитектуру клиент-серверного приложения и структуру базы данных.
4. Реализовать серверную часть приложения с использованием FastAPI и обеспечить обработку запросов пользователей.
5. Разработать клиентский интерфейс с использованием HTML, CSS и JavaScript.
6. Реализовать CRUD-операции для работы с мероприятиями и пользователями, а также систему разграничения прав доступа.
7. Провести тестирование разработанной системы и оценить её эффективность.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Разрабатываемое веб-приложение предназначено для автоматизации процессов документооборота и управления мероприятиями в культурно-досуговых учреждениях. Основными пользователями системы являются сотрудники учреждения, участвующие в организации, согласовании и сопровождении мероприятий.

В системе выделяются следующие основные роли пользователей:

- **Сотрудник** — базовый пользователь системы, осуществляющий работу с карточками мероприятий, документами и отчетностью.
- **Руководитель СП** — пользователь, ответственный за создание мероприятий, назначение ответственных сотрудников и контроль выполнения задач. Руководитель структурного подразделения.
- **Художественный руководитель** — пользователь, выполняющий согласование или отклонение мероприятий, а также формирование статистики и отчетности.
- **ИТ-администратор** — пользователь, осуществляющий управление учетными записями, ролями и правами доступа.

При проектировании интерфейса учитывались особенности работы сотрудников культурно-досуговых учреждений: необходимость быстрого доступа к документам, минимизация количества действий при работе с мероприятиями, а также простота освоения системы без длительного обучения.

2.2 Описание функциональности приложения

Функциональность системы определяется процессами создания, согласования и сопровождения мероприятий, а также хранением связанной документации.

2 Основных сценария использования системы могут быть представлены в виде следующих User Story:

«Я, как руководитель структурного подразделения, хочу создавать карточки мероприятий, указывать их категорию, дату проведения, ответственного сотрудника и плановые показатели, чтобы информация о мероприятиях хранилась централизованно и могла использоваться для последующего формирования отчётности».

«Я, как художественный руководитель, хочу просматривать сведения о проведённых мероприятиях, количестве посетителей и полученных внебюджетных средствах за выбранный период, чтобы оперативно формировать ежемесячные отчёты и контролировать результаты работы учреждения».

Система предоставляет следующие функциональные возможности:

- авторизация пользователей;
- создание и редактирование карточек мероприятий;
- назначение ответственных сотрудников;
- согласование и отклонение мероприятий;
- загрузка и хранение документов;
- формирование статистики и отчетности;
- выгрузка документов и отчетов;
- управление учетными записями и ролями пользователей.

На рисунке 2.13 представлена диаграмма вариантов использования системы.

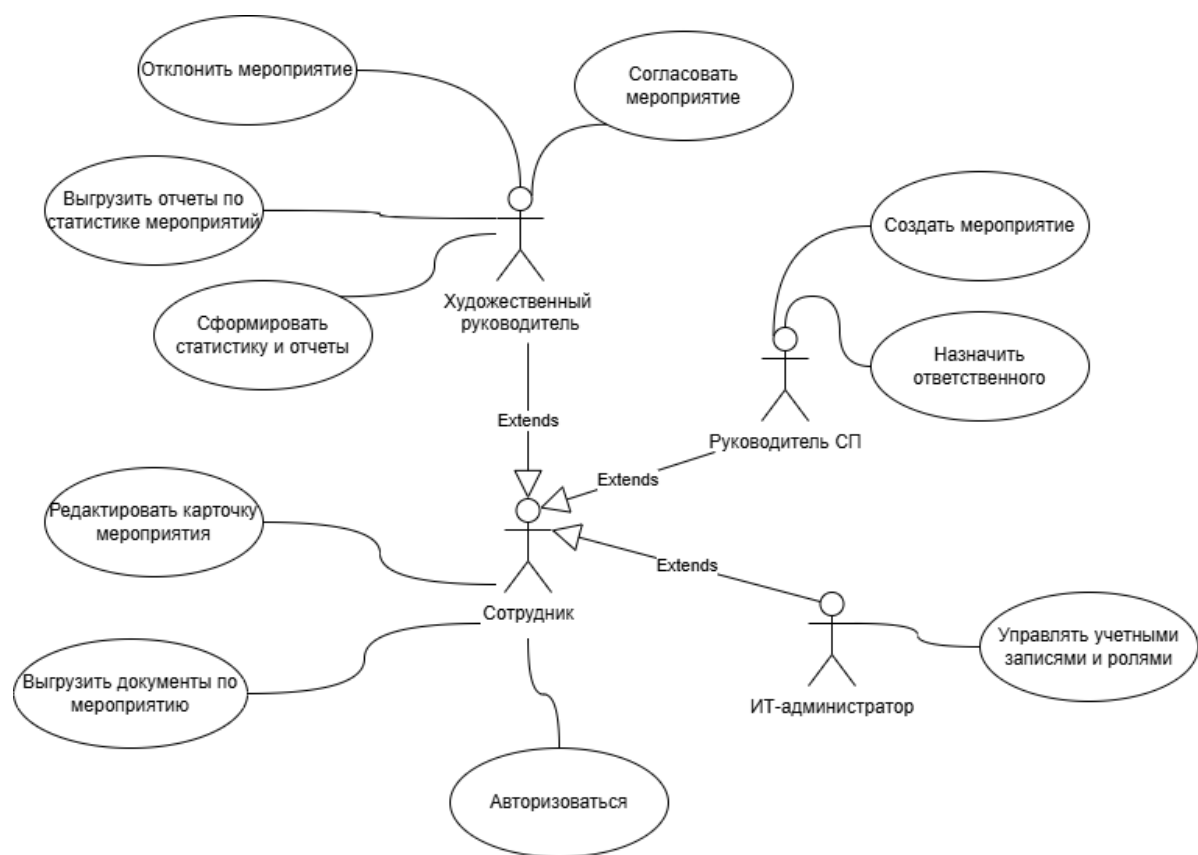


Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

Для хранения данных приложения выбрана реляционная модель данных на основе СУБД PostgreSQL. Использование реляционной базы данных позволяет обеспечить целостность данных, поддержку связей между сущностями и разграничение прав доступа пользователей.

Основными сущностями системы являются:

- пользователи;
- роли;
- мероприятия;
- категории мероприятий;
- статусы мероприятий;
- файлы.

Центральной сущностью системы является мероприятие, содержащее основную информацию о событии, статусе согласования, категории, ответственном пользователе и связанных документах.

На рисунке 2.2 представлена ER-диаграмма базы данных.

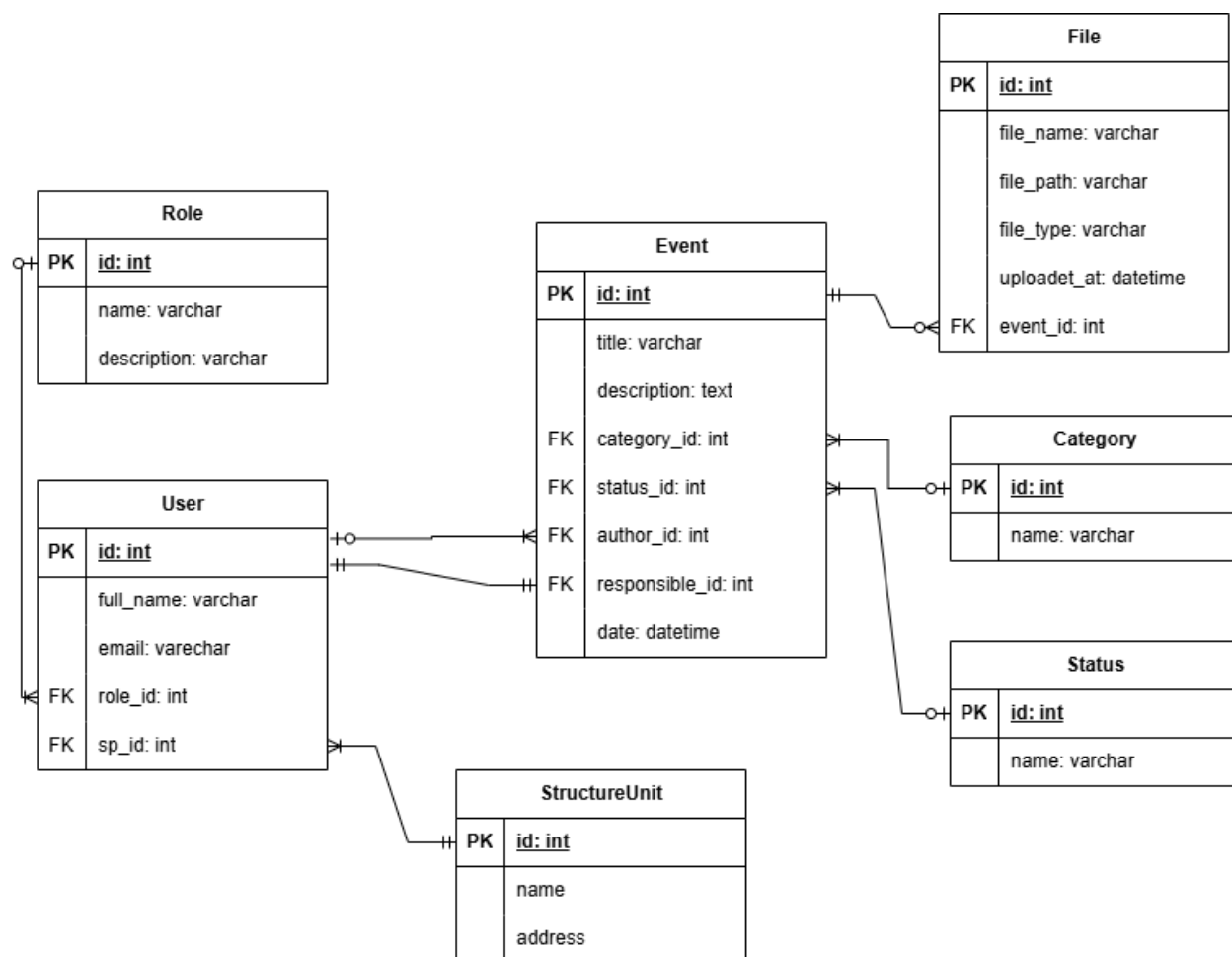


Рисунок 2.2 – ER-диаграмма базы данных

Структура основной таблицы мероприятий представлена в таблице 2.1.

Таблица 2.1 – Структура таблицы мероприятий (event)

Поле	Тип данных	Описание
id	INT	Уникальный идентификатор мероприятия
title	VARCHAR	Название мероприятия
description	TEXT	Описание мероприятия
category_id	INT	Ссылка на категорию мероприятия
status_id	INT	Ссылка на статус мероприятия
author_id	INT	Идентификатор автора мероприятия

Поле	Тип данных	Описание
responsible_id	INT	Идентификатор ответственного сотрудника
date	DATETIME	Дата проведения мероприятия

2.4 Разработка макетов страниц

На этапе проектирования интерфейса были разработаны макеты основных страниц веб-приложения, отражающие ключевые пользовательские сценарии.

Основное внимание при проектировании интерфейса уделялось:

- простоте навигации;
- минимизации количества действий пользователя;
- удобству работы с документами и мероприятиями;
- единообразию элементов интерфейса;
- адаптации интерфейса под различные роли пользователей.

В системе предусмотрены следующие основные страницы:

- страница авторизации;
- список мероприятий;
- форма создания мероприятия;
- карточка мероприятия;
- страница управления пользователями;
- страница отчетности.

Макеты страниц разрабатывались с помощью простой верстки HTML+CSS.

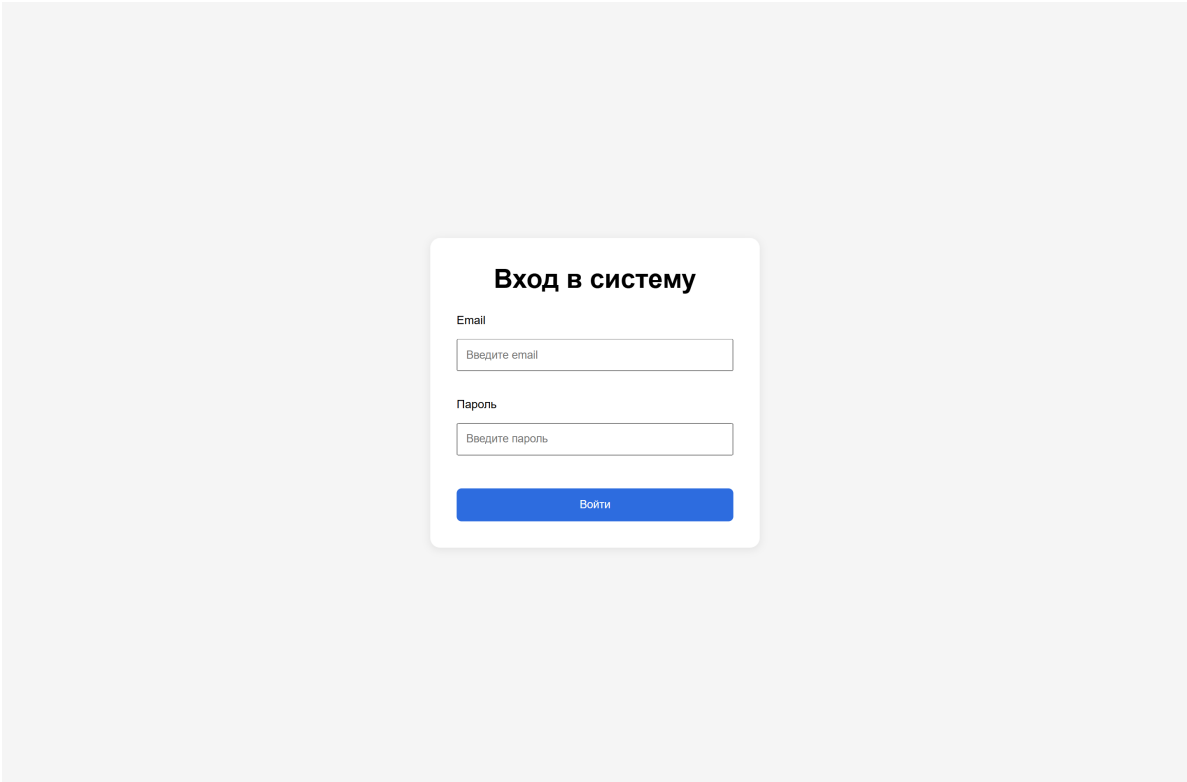


Рисунок 2.3 – Страница авторизации

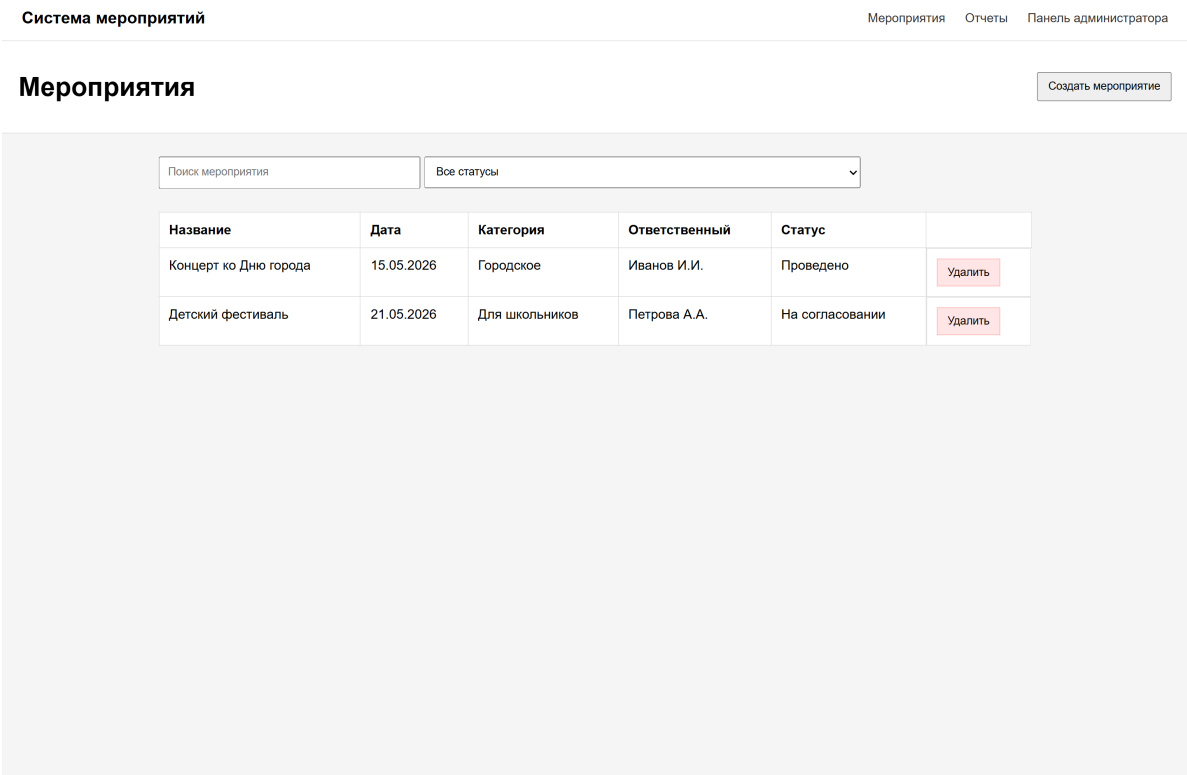


Рисунок 2.4 – Главная страница со списком мероприятий

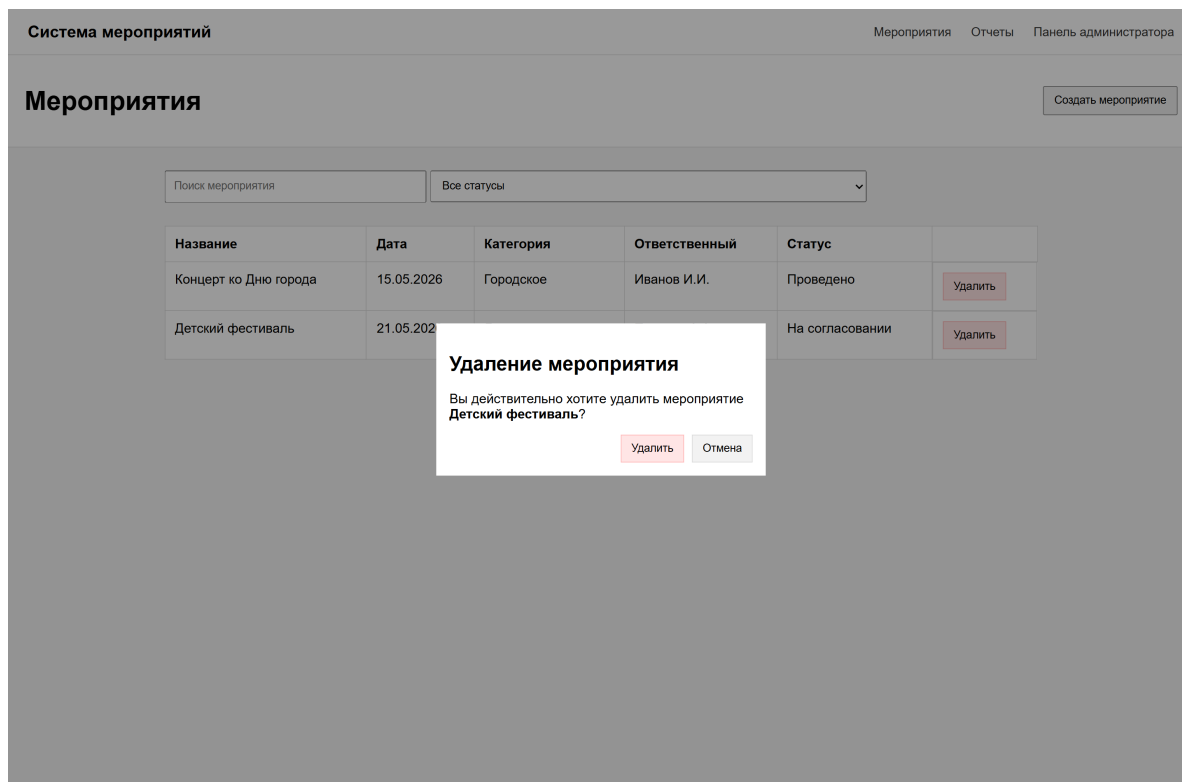


Рисунок 2.5 – Модальное окно удаления мероприятия

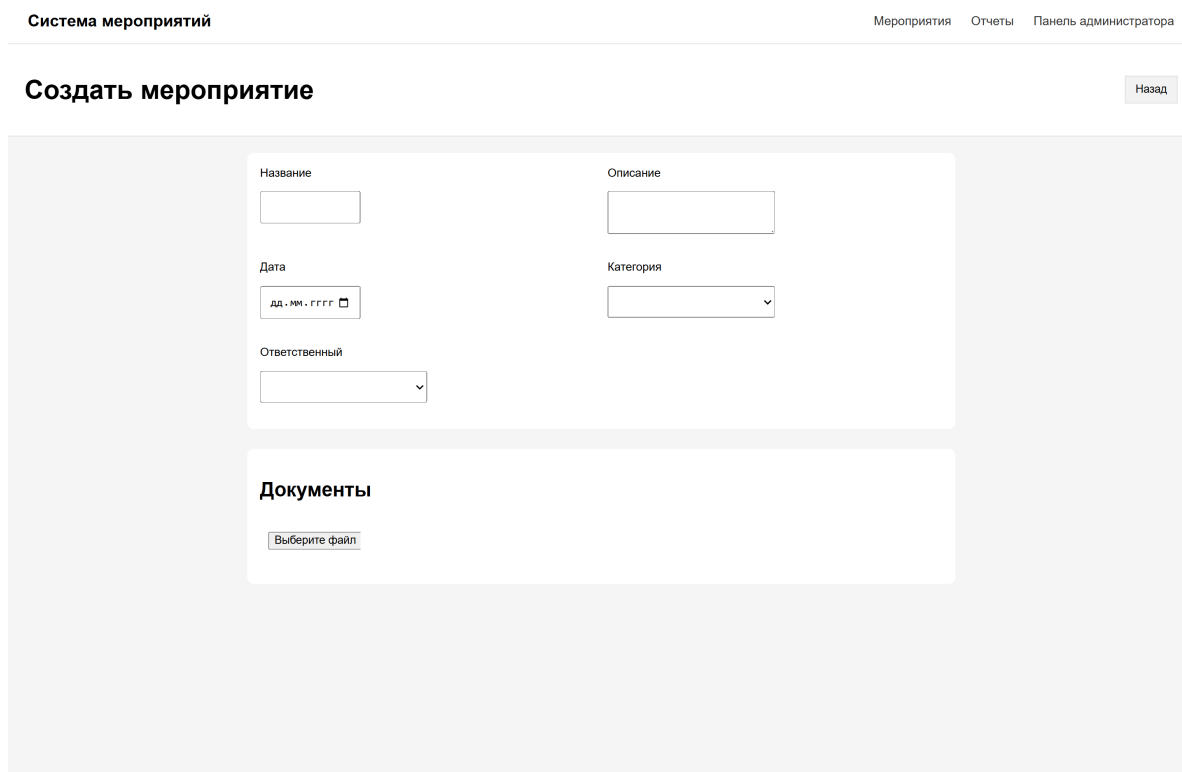


Рисунок 2.6 – Страница создания мероприятия

Карточка мероприятия

Редактировать

Назад

Название	Дата проведения
Концерт ко Дню города	15.09.2026
Описание	
Праздничное мероприятие для жителей города с концертной программой, интерактивными зонами и выступлением творческих коллективов.	
Категория	Ответственный сотрудник
Городское	Иванов И.И.

Документы

Файл	Тип	Действие
Афиша.pdf	PDF	Скачать
Смета.xlsx	XLSX	Скачать

Рисунок 2.7 – Страница просмотра и редактирования мероприятия

Управление пользователями и СП

Добавить пользователя

Добавить СП

Пользователи

ФИО	Email	Должность	СП	Действия
Иванов И.И.	ivanov@mail.ru	Руководитель СП	Ретро	Редактировать Удалить
Петрова А.А.	petrova@mail.ru	Художественный руководитель	Отдел краеведения	Редактировать Удалить

Структурные подразделения

Название	Адрес
Ретро	г. Фрязино, ул. Полевая, д. 6
Отдел краеведения	г. Фрязино, ул. Комсомольская, д. 28

Рисунок 2.8 – Панель администратора

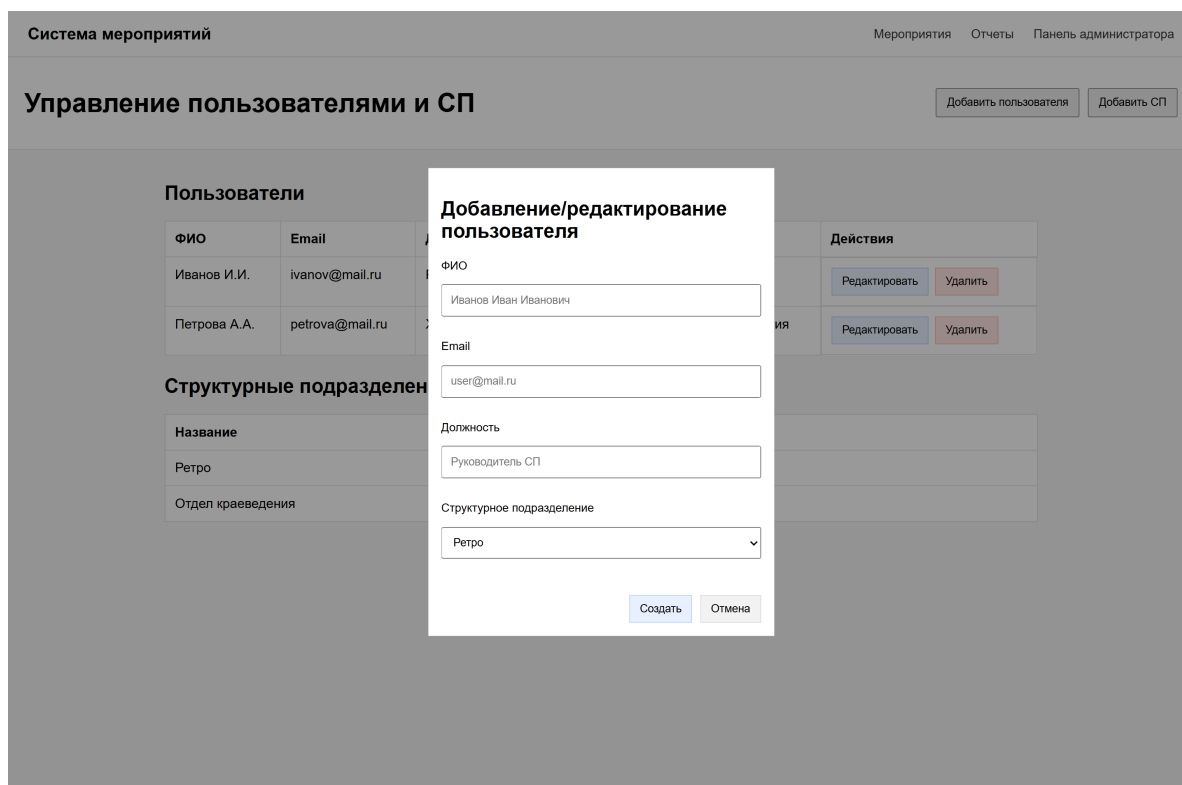


Рисунок 2.9 – Панель администратора: добавление или редактирование пользователя

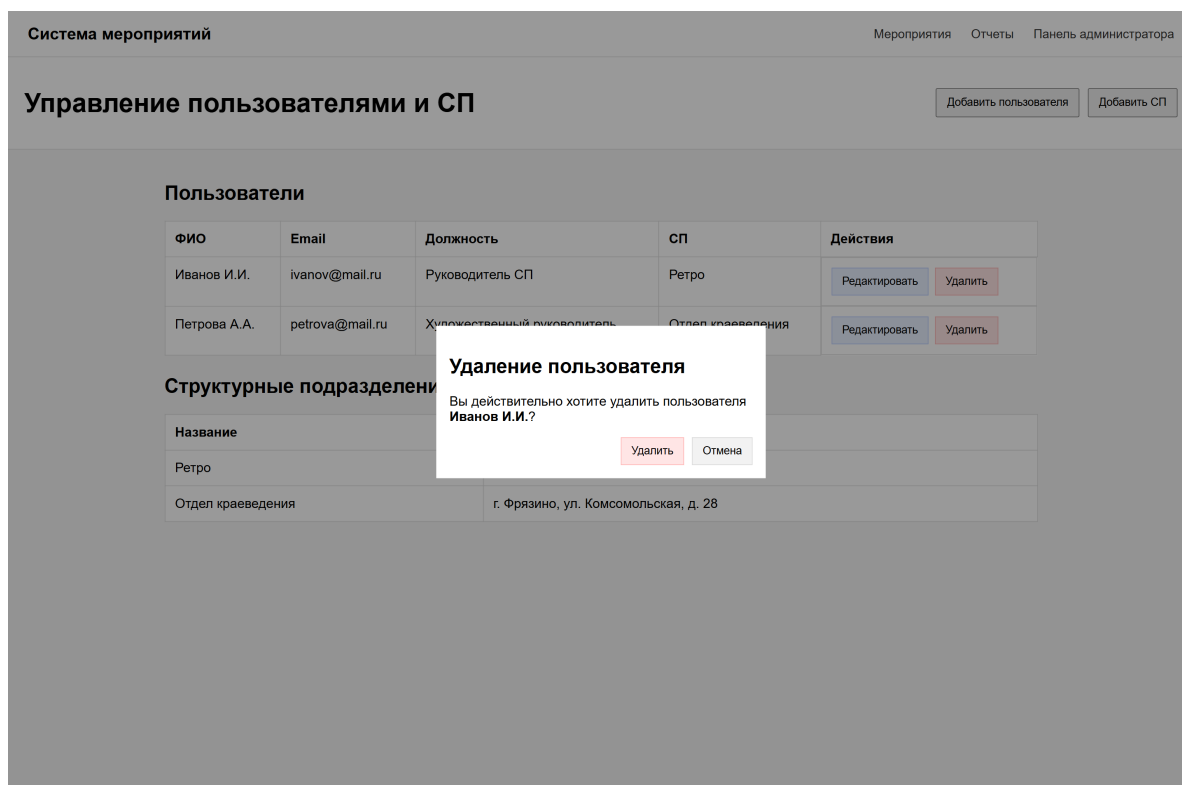


Рисунок 2.10 – Панель администратора: удаление пользователя

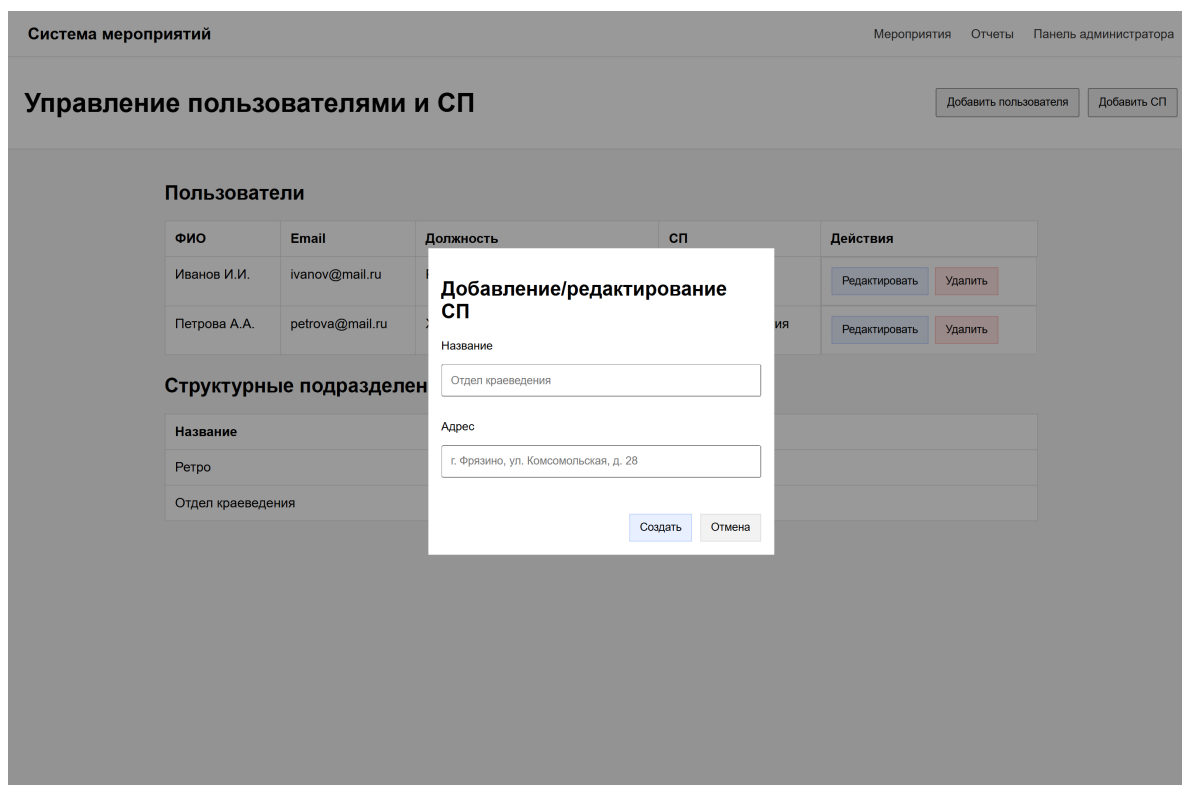


Рисунок 2.11 – Панель администратора: добавление или редактирование структурного подразделения

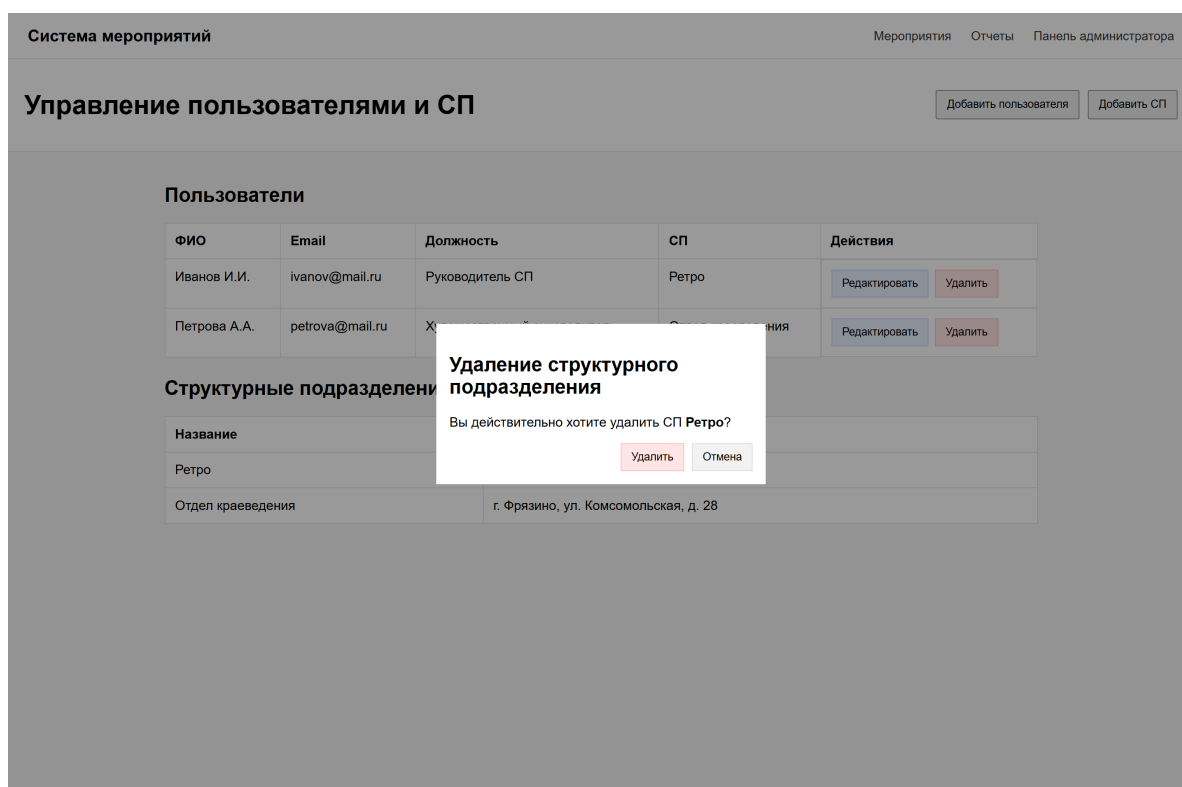


Рисунок 2.12 – Панель администратора: удаление структурного подразделения

Отчетность и статистика

Мероприятий за месяц

48

Формирование отчетов

Сформировать месячный отчет

Сформировать отчет по сотрудникам

Сформировать отчет по СП

Рисунок 2.13 – Страница выгрузки отчетов

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Разработка пользовательского интерфейса и реализация базовой структуры страниц

В соответствии с задачей 3.1 была реализована серверная шаблонизация на основе движка Jinja2, встроенного в фреймворк FastAPI. Все страницы приложения строятся через наследование от единого базового шаблона `base.html`, что обеспечивает единообразие навигации и стилей.

Структура базового шаблона содержит:

- общую шапку с логотипом системы и навигационным меню;
- блоки `{block content}`, `{block modals}` и `{block scripts}`, переопределяемые на дочерних страницах;
- подключение глобальной таблицы стилей `style.css`.

Навигационное меню формируется динамически в зависимости от роли вошедшего пользователя: администратор видит ссылку на панель управления, художественный руководитель — дополнительный пункт «Отчеты», остальные роли получают только доступ к разделу мероприятий. Пример логики условного отображения меню приведён в листинге 1.

Листинг 1 – Фрагмент шаблона навигационного меню

```
1 <nav class="main-nav">
2     {% if current_user.role == 'ADMIN' %}
3         <a href="/adminПанель"> администратора</a>
4     {% else %}
5         <a href="/eventsМероприятия"></a>
6         {% if current_user.role == 'ART_DIRECTOR' %}
7             <a href="/reportsОтчеты"></a>
8         {% endif %}
9     {% endif %}
10    <a href="/logoutВыйти"></a>
11 </nav>
```

Корневой маршрут / перенаправляет пользователя на страницу мероприятий с помощью `RedirectResponse`, что позволяет избежать пустой стартовой страницы. Статические файлы (шрифты, CSS) обслуживаются через механизм `StaticFiles` фреймворка FastAPI по префиксу `/static`.

Для единообразия визуального оформления в приложении используется фирменный шрифт «George» в четырёх начертаниях (Regular, SemiBold, Bold, Italic). Цветовая палитра и шрифт соответствуют фирменному стилю ДК Подмосковья [4]. На страницах со списками мероприятий реализована постраничная навигация (пагинация): число записей на странице фиксировано, а ссылки на соседние страницы формируются шаблонным тегом с передачей текущих параметров фильтрации (см. листинг 2).

Листинг 2 – Фрагмент шаблона пагинации

```
1 {% if total_pages > 1 %}
2 <nav class="pagination">
3   {% if page > 1 %}
4     <a href="?search={{ search }}&status={{ status_filter }}
5       &page={{ page - 1 }}" class="page-btn">&laquo
6     ;</a>
7   {% endif %}
8   {% for p in range(1, total_pages + 1) %}
9     <a href="?search={{ search }}&status={{ status_filter
10      }}&page={{ p }}"
11      class="page-btn {% if p == page %}active{% endif
12      %}">{{ p }}</a>
13   {% endfor %}
14   {% if page < total_pages %}
15     <a href="?search={{ search }}&status={{ status_filter }}
16       &page={{ page + 1 }}" class="page-btn">&raquo;
17     ;</a>
18   {% endif %}
19 </nav>
20 {% endif %}
```

3.2 Реализация модуля аутентификации пользователей и разграничения прав доступа

Согласно задаче 3.2, в системе реализована многоуровневая модель доступа на основе ролей (RBAC). Предусмотрены четыре роли:

- ADMIN — системный администратор, управляет учётными записями пользователей;
- ART_DIRECTOR — художественный руководитель, имеет полный доступ к мероприятиям и отчётам;
- DEPARTMENT_HEAD — руководитель отдела, создаёт мероприятия и назначает ответственных;
- EMPLOYEE — сотрудник, редактирует мероприятия, за которые назначен ответственным.

Для обеспечения безопасности применяется протокол JWT (JSON Web Token) в связке с библиотекой python-jose. При успешной аутентификации сервер генерирует токен, содержащий идентификатор пользователя в поле sub и временную метку истечения exp. Токен подписывается секретным ключом по алгоритму HS256. (см. листинг 3).

Листинг 3 – Функция генерации JWT-токена

```
1 def create_access_token(  
2     subject: str,  
3     expires_delta: timedelta | None = None,  
4 ) -> str:  
5     expire = datetime.now(UTC) + (  
6         expires_delta if expires_delta is not None  
7         else timedelta(minutes=settings.  
8             jwt_access_token_expire_minutes)  
9     )  
10    payload: dict[str, Any] = {"sub": subject, "exp": expire  
11    }  
12    return jwt.encode(  
13        payload,  
14        settings.jwt_secret_key,  
15        algorithm=settings.jwt_algorithm,  
16    )
```


Пароли хранятся в базе данных в хешированном виде с использованием библиотеки `bcrypt`, которая автоматически добавляет «соль» при каждом вызове `hashpw`. Это исключает атаки по заранее вычисленным таблицам (rainbow tables).

Для верификации входящих запросов зависимость `get_current_user` (модуль `app/api/dependencies.py`) извлекает токен из куки `access_token`, декодирует его, получает из поля `sub` идентификатор пользователя и загружает соответствующую запись из базы данных. Если токен недействителен или пользователь не найден, возвращается HTTP-ответ 401 Unauthorized.

Разграничение прав доступа к операциям над мероприятиями реализовано в модуле `app/core/permissions.py` через набор чистых функций (листинг 4).

Листинг 4 – Функции проверки прав доступа

```
1 def can_create_event(user: User) -> bool:
2     return user.role in (Role.DEPARTMENT_HEAD, Role.
    ART_DIRECTOR)
3
4 def can_edit_event(user: User, event: Event) -> bool:
5     if user.role == Role.ART_DIRECTOR:
6         return True
7     if user.role in (Role.EMPLOYEE, Role.DEPARTMENT_HEAD):
8         return event.responsible_id == user.id
9     return False
10
11 def can_delete_event(user: User) -> bool:
12     return user.role == Role.ART_DIRECTOR
13
14 def can_assign_responsible(user: User) -> bool:
15     return user.role in (Role.DEPARTMENT_HEAD, Role.
    ART_DIRECTOR)
```

Функции используются непосредственно в обработчиках API: при нарушении правил доступа возвращается ответ 403 Forbidden с текстом «Недостаточно прав».

3.3 Разработка CRUD-интерфейсов для управления мероприятиями и документами

В соответствии с задачей 3.3 разработаны полные CRUD-интерфейсы для двух ключевых сущностей: мероприятий и прикрепленных документов.

Управление мероприятиями

Модель Event (таблица events) содержит следующие поля:

- title — название мероприятия (VARCHAR(255));
- description — текстовое описание (TEXT, необязательное);
- category_id — внешний ключ на справочник категорий;
- status — текущий статус из перечисления EventStatus;
- author_id — создатель записи;
- responsible_id — назначенный ответственный (может быть NULL);
- planned_date — планируемая дата проведения (TIMESTAMP WITH TIME ZONE);
- created_at, updated_at — временные метки создания и последнего изменения.

REST API для работы с мероприятиями реализован в модуле app/api/events.py. Поддерживаются следующие операции:

- GET /api/v1/events — получить список всех мероприятий с возможностью фильтрации по статусу и поиску по названию;
- GET /api/v1/events/{id} — получить одно мероприятие по идентификатору;
- POST /api/v1/events — создать новое мероприятие (только для ролей DEPARTMENT_HEAD и ART_DIRECTOR);
- PATCH /api/v1/events/{id} — частичное обновление полей мероприятия;

- DELETE /api/v1/events/{id} — удалить мероприятие (только ART_DIRECTOR).

Обработчик создания мероприятия приведён в листинге 5.

Листинг 5 – Обработчик создания мероприятия

```
1 @router.post("", response_model=EventResponse ,
2             status_code=status.HTTP_201_CREATED)
3 async def create_event(
4     data: EventCreate ,
5     session: AsyncSession = Depends(get_db),
6     current_user: User = Depends(get_current_user),
7 ) -> Event:
8     if not can_create_event(current_user):
9         _403()
10
11     responsible_id = data.responsible_id
12     if responsible_id is not None and not
13     can_assign_responsible(current_user):
14         responsible_id = None
15
16     event = await EventRepository(session).create(
17         title=data.title,
18         description=data.description,
19         category_id=data.category_id,
20         author_id=current_user.id,
21         planned_date=data.planned_date,
22         responsible_id=responsible_id,
23     )
24     await session.commit()
25     return event
```

На стороне фронтенда список мероприятий отображается в виде таблицы с колонками «Название», «Дата», «Категория», «Ответственный» и «Статус». Кнопки «Изменить» и «Удалить» отображаются только у тех строк, к которым у текущего пользователя есть соответствующие права.

Управление прикреплёнными документами

Для каждого мероприятия предусмотрена возможность прикрепления произвольных файлов (документов, изображений и т.д.). Модель File хранит путь до файла на диске, оригинальное имя, MIME-тип и временную метку

загрузки. Загруженные файлы сохраняются в директории uploads/{event_id}/ с уникальным именем на основе UUID, что исключает конфликты имён.

REST API работы с файлами (app/api/files.py) включает:

- POST /api/v1/events/{id}/files — загрузка файла (доступна ответственно-му сотруднику или арт-директору);
- GET /api/v1/files/{id}/download — скачивание файла по идентификатору;
- DELETE /api/v1/files/{id} — удаление файла (физическое удаление с диска и из базы данных).

На странице карточки мероприятия файлы отображаются в виде таблицы, где каждое имя является ссылкой для скачивания. Загрузка новых файлов выполняется асинхронно через Fetch API без перезагрузки страницы (листинг 6).

Листинг 6 – JavaScript-функция загрузки файлов

```
1 async function uploadFiles() {
2     const input = document.getElementById('file-upload');
3     if (!input || !input.files.length) {
4         alertВыберите(' файлы'); return;
5     }
6     let hasError = false;
7     for (const file of input.files) {
8         const fd = new FormData();
9         fd.append('file', file);
10        const r = await fetch(
11            '/api/events/{ { event.id } }/files',
12            { method: 'POST', body: fd }
13        );
14        if (!r.ok) hasError = true;
15    }
16    if (hasError) alertНе(' удалось загрузить файлы');
17    location.reload();
18 }
```

3.4 Реализация механизма формирования и выгрузки отчётов в формате Excel

В соответствии с задачей 3.4 реализован механизм генерации отчётов в формате Excel (.xlsx). Для работы с табличными данными используется библиотека `pandas`, а для записи файлов — `openpyxl`.

Все конечные точки отчётности расположены в модуле `app/api/reports.py`. Ответ каждого эндпоинта является потоковым (`StreamingResponse`) с правильно установленными заголовками `Content-Disposition` и `Content-Type`, что обеспечивает автоматическое скачивание файла браузером (листинг 7).

Листинг 7 – Вспомогательная функция формирования Excel-ответа

```
1 def _xlsx_response(buf: io.BytesIO, filename: str) ->
    StreamingResponse:
2     buf.seek(0)
3     return StreamingResponse(
4         buf,
5         media_type=(
6             "application/vnd.openxmlformats-"
7             "officedocument.spreadsheetml.sheet"
8         ),
9         headers={
10             "Content-Disposition":
11                 f'attachment; filename="{filename}"',
12         },
13     )
```

Месячный отчёт

Эндпоинт `GET /api/v1/reports/monthly` формирует сводку за предыдущий календарный месяц. С помощью функций `extract SQLAlchemy` выполняется выборка мероприятий, чья дата проведения попадает в нужный период. Результирующий файл содержит два листа:

- «Итог (месяц год)» — сводная таблица: количество проведённых и запланированных мероприятий, самая частая категория;
- «По категориям» — детализация проведённых мероприятий в разрезе категорий.

Отчёт по сотрудникам

Эндпоинт GET /api/v1/reports/employees агрегирует данные по всем сотрудникам (роли, отличные от ADMIN). Для каждого сотрудника рассчитываются:

- количество созданных мероприятий;
- количество проведённых мероприятий (в роли ответственного);
- доля отклонённых мероприятий от общего числа созданных, %;
- доля незавершённых мероприятий (статус PLANNED) от суммы запланированных и проведённых, %.

Пример вычисления процента отклонённых мероприятий приведён в листинге 8.

Листинг 8 – Вычисление доли отклонённых мероприятий

```
1 total_by_author = (  
2     df_events.groupby("author_id").size()  
3     .reset_index(name="total")  
4     .rename(columns={"author_id": "id"})  
5 )  
6 rejected_by_author = (  
7     df_events[df_events["status"] == rejected_str]  
8     .groupby("author_id").size()  
9     .reset_index(name="rejected")  
10    .rename(columns={"author_id": "id"})  
11 )  
12 rej = total_by_author.merge(  
13     rejected_by_author, on="id", how="left"  
14 ).fillna(0)  
15 rej["% отклоненных"] = (  
16     rej["rejected"] / rej["total"] * 100  
17 ).round(1)
```

Файл также содержит второй лист «По категориям» с разбивкой проведённых мероприятий каждого сотрудника по категориям в виде сводной (pivot) таблицы.

Отчёт по структурным подразделениям

Эндпоинт GET /api/v1/reports/structure-units формирует сводку в разрезе структурных подразделений (СП). Для каждого подразделения определяются:

- количество сотрудников;
- суммарное количество проведённых мероприятий, ответственный за которые числится в данном СП;
- разбивка проведённых мероприятий по категориям.

На клиентской стороне скачивание файла инициируется JavaScript: браузер получает Blob через Fetch API и создаёт временную ссылку с атрибутом download, после чего программно выполняет клик (листинг 9).

Листинг 9 – Скачивание отчёта на стороне клиента

```
1 async function downloadReport(type) {
2     const resp = await fetch(`/api/reports/${type}`);
3     const blob = await resp.blob();
4     const disposition =
5         resp.headers.get('Content-Disposition') || '';
6     const match = disposition.match(/filename="([^"]+)"/);
7     const filename = match
8         ? match[1]
9         : `report_${type}.xlsx`;
10    const url = URL.createObjectURL(blob);
11    const a = document.createElement('a');
12    a.href = url; a.download = filename; a.click();
13    URL.revokeObjectURL(url);
14 }
```

3.5 Разработка системы мониторинга статусов мероприятий и формирования отчётности

Согласно задаче 3.5, в системе реализован жизненный цикл мероприятия на основе конечного автомата (State Machine). Допустимые переходы между статусами описаны в виде словаря `_TRANSITIONS` в модуле `app/core/permissions.py` и представлены на схеме 10.

Листинг 10 – Таблица допустимых переходов статусов

```
1 _TRANSITIONS: dict[EventStatus, set[EventStatus]] = {
2     EventStatus.ON_APPROVAL: {
3         EventStatus.APPROVED, EventStatus.REJECTED
4     },
5     EventStatus.APPROVED: {EventStatus.PLANNED},
6     EventStatus.PLANNED: {EventStatus.COMPLETED},
7     EventStatus.COMPLETED: set(),
8     EventStatus.REJECTED: set(),
9 }
```

Перевод мероприятия в статусы `APPROVED` и `REJECTED` доступен исключительно художественному руководителю (`ART_DIRECTOR`). Перевод из `PLANNED` в `COMPLETED` выполняет ответственный сотрудник или арт-директор.

При каждой смене статуса в таблицу `event_history` автоматически добавляется запись с указанием пользователя, совершившего изменение, предыдущего и нового статуса, а также необязательного комментария (листинг 11).

Листинг 11 – Обработчик изменения статуса мероприятия

```
1 @router.post("/{event_id}/status",
2             response_model=EventResponse)
3 async def change_status(
4     event_id: int,
5     data: EventStatusUpdate,
6     session: AsyncSession = Depends(get_db),
7     current_user: User = Depends(get_current_user),
8 ) -> Event:
9     event = await _get_event_or_404(event_id, session)
10
11     if not can_change_status(current_user, event, data.
12                               status):
13         _403()
14
15     old_status = event.status
16     event = await EventRepository(session).update(
17         event, status=data.status
18     )
19     await EventHistoryRepository(session).create(
20         event_id=event.id,
21         user_id=current_user.id,
22         old_status=old_status,
23         new_status=data.status,
24         comment=data.comment,
25     )
26     await session.commit()
27     return event
```

На странице карточки мероприятия отображается:

- текущий статус в виде цветного значка-бейджа;
- выпадающий список доступных переходов (формируется функцией `allowed_statuses` на основе роли пользователя и текущего статуса);
- поле для комментария при смене статуса;
- таблица истории изменений со временем, автором и комментарием каждого перехода.

Страница отчётности (`/reports`), доступная исключительно художественному руководителю, выводит счётчик проведённых мероприятий и три кнопки

для загрузки отчётов: месячного, по сотрудникам и по структурным подразделениям. Запросы к API выполняются асинхронно, а пользователь информируется о процессе формирования файла через строку статуса под кнопками.

Фильтрация списка мероприятий на главной странице позволяет осуществлять оперативный мониторинг: пользователь может отобразить записи по любому из пяти статусов (*На согласовании*, *Согласовано*, *Отклонено*, *Запланировано*, *Проведено*) или выполнить текстовый поиск по названию мероприятия, что обеспечивает быстрый доступ к актуальным сведениям о ходе выполнения культурных мероприятий учреждения.

Репозиторий с кодом проекта: https://github.com/xseniaKri/coursework_rvp

СПИСОК ЛИТЕРАТУРЫ

- [1] MDN Web Docs [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://developer.mozilla.org/> (дата обращения: 27.04.2026).
- [2] FastAPI Documentation [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://fastapi.tiangolo.com/> (дата обращения: 27.04.2026).
- [3] PostgreSQL Documentation [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://www.postgresql.org/docs/> (дата обращения: 27.04.2026).
- [4] Руководство по использованию фирменного стиля, ДК Подмосковья [Электронный ресурс]. — [Б. м. : б. и.]. — URL: <https://mk.mosreg.ru/download/document/17467318> (дата обращения: 06.06.2026).